

Unity 스크립트로서의 C#

일반 C# 프로그램과 Unity 스크립트의 차이

강영민

2026년 1학기 | Lecture 9

동명대학교 게임학부

목차

이번 강의의 출발점

Unity 스크립트의 기본 구조

MonoBehaviour의 생명 주기

컴포넌트 기반 설계

Inspector와 직렬화

주요 고려 사항

Lecture 7과의 비교

실전 예제: 충돌하면 합쳐지는 공

단계별 도전 과제

정리

이번 강의의 출발점

Lecture 7에서 우리가 직접 한 일들

- `Application.Run(new GameForm())` — 진입점
- `Timer.Tick`으로 16ms 간격 게임 루프
- `Invalidate()`로 다시 그리기 요청
- `OnPaint`에서 `Graphics`로 직접 그리기
- `OnKeyDown/OnKeyUp`으로 입력 추적
- `Math.Abs`로 수동 충돌 판정

Unity에서는 이 모든 것이 이미 만들어져 있다.

우리는 `Update()` 같은 약속된 메서드만 채워 넣는다.

호출 방향의 역전

- 일반 C# 프로그램: **우리가** 라이브러리를 호출
`Console.WriteLine(...)`
- Unity 스크립트: **프레임워크가** 우리를 호출
`void Update() { ... } ← Unity가 자동 호출`

이 “역전”을 **IoC(Inversion of Control)** 또는
헐리우드 원칙(“Don't call us, we'll call you”)이라 한다.

Unity 스크립트의 기본 구조

가장 기본적인 Unity 스크립트

```
using UnityEngine;

public class HelloUnity : MonoBehaviour
{
    void Start()
    {
        Debug.Log("Hello, Unity!");
    }

    void Update()
    {
        // Called every frame
    }
}
```

- using UnityEngine; — Unity 기본 네임스페이스
- : MonoBehaviour — Unity가 인식하는 “스크립트 컴포넌트”
- 클래스 이름 = 파일 이름이어야 함

MonoBehaviour란 정확히 무엇인가?

Unity가 미리 만들어 둔 **기반 클래스**로, 다음을 가짐:

- 미리 정의된 “빈” 메서드들

Awake, Start, Update, OnCollisionEnter ...

- 유용한 속성들

transform, gameObject, enabled

Lecture 7과 비교

class GameForm : Form으로 OnPaint를 override했던 것과 같은 구조.

단, Unity는 **override** 키워드를 쓰지 않는다.

⇒ 메서드 이름 철자가 틀리면 그냥 호출되지 않을 뿐 컴파일 에러는 안 남.

Main이 없다는 게 무슨 뜻인가?

Unity Editor에서 **재생(Play)** 버튼을 누르면:

1. 현재 **Scene**의 모든 GameObject를 찾는다
2. 각 GameObject에 붙은 MonoBehaviour를 찾는다
3. Awake → OnEnable → Start 호출
4. 매 프레임 모든 활성 스크립트의 Update 호출

Unity 엔진 자체가 거대한 **Main**이라고 생각하면 된다.

Scene(씬)이란?

“한 화면을 구성하는 GameObject들의 모음”

- TitleScene — 타이틀 화면
- StageScene — 게임 플레이 화면
- EndingScene — 엔딩 화면

파일로 저장됨 (.unity).

`SceneManager.LoadScene("StageScene")`으로 전환.

Lecture 7: 하나의 Form만 띄움

Unity: 여러 Scene을 갈아 끼움 → 화면별 코드 분리 용이

MonoBehaviour의 생명 주기

메서드	호출 시점
Awake()	생성될 때 한 번 (비활성도 호출)
OnEnable()	활성화될 때마다
Start()	첫 Update 직전, 한 번
Update()	매 프레임 (보통 30~120fps)
FixedUpdate()	고정 시간 간격 (0.02초)마다 — 물리
LateUpdate()	모든 Update 후 — 카메라 추적
OnDisable()	비활성화될 때
OnDestroy()	삭제 직전

Awake vs Start — 둘 다 “초기화”인데?

Scene에 A, B, C 세 오브젝트가 있을 때 호출 순서:

1. A.Awake → B.Awake → C.Awake
(모든 오브젝트가 “기본 준비” 끝)
2. A.Start → B.Start → C.Start
(다른 오브젝트가 이미 Awake되어 있음이 보장)
3. Update 루프 시작

규칙

- Awake: 자기 자신의 변수 초기화, 컴포넌트 캐싱
- Start: 다른 오브젝트와 상호작용하는 초기화

Update vs FixedUpdate

	Update	FixedUpdate
호출 간격	프레임마다 (불규칙)	고정 (0.02초)
용도	입력, 일반 로직	Rigidbody 물리
시간 변수	<code>Time.deltaTime</code>	<code>Time.fixedDeltaTime</code>

- 키 입력 검사 → Update
- `transform.position` 직접 변경 → Update
- `Rigidbody.AddForce` → FixedUpdate
- 카메라 추적 → LateUpdate
(캐릭터 Update가 끝난 후 따라가야 끊김 없음)

“이 오브젝트는 물리 법칙의 영향을 받는다”

- 중력 — 자유 낙하
- 관성 — 마찰 없는 한 계속 운동
- 충돌 반응 — Collider와 부딪히면 튕김

Lecture 7의 `if (x < 0 || x > 760) dx = -dx;`

→ Unity에서는 한 줄도 안 쓰고 자동으로 됨.

왜 FixedUpdate? → 고정 간격이 아니면 PC 성능에 따라 물리 결과가 달라짐.

```
void Update()
{
    // BAD: speed depends on frame rate
    transform.position += new Vector3(0.1f, 0, 0);

    // GOOD: 5 units per second, regardless of fps
    transform.position += new Vector3(
        5f * Time.deltaTime, 0, 0);
}
```

- 60fps $\rightarrow$ Time.deltaTime $\approx$ 0.0167s
- 30fps $\rightarrow$ Time.deltaTime $\approx$ 0.0333s
- 이동량 = 속도 $\times$ Time.deltaTime $\rightarrow$ 프레임 무관 일정 속도

컴포넌트 기반 설계

Lecture 7 vs Unity — 객체 만드는 방식

Lecture 7 (상속)

```
class Player {  
    public float X, Y;  
    void Update(...) { ... }  
    void Draw(Graphics g) { ... }  
}
```

Unity (조합)

```
GameObject "Player"  
+ Transform          (position, rotation, scale)  
+ SpriteRenderer     (image)  
+ Rigidbody2D        (physics)  
+ BoxCollider2D      (collision shape)  
+ PlayerController   (our script)
```

상속(Inheritance) vs 조합(Composition)

상속 = “무엇이다” (is-a)

- Player는 FlyingCharacter이다
→ FlyingCharacter는 Movable이다 ...
- 새 기능 추가 = 새 자식 클래스 → 상속 지옥

조합 = “무엇을 가진다” (has-a)

- GameObject가 Mover, Shooter, HealthComponent를 가짐
- “걷는 적” 필요 → 적에 Mover만 붙이면 끝
- 레고 블록처럼 기능을 조립

Unity는 의도적으로 조합을 선택했다.

GetComponent — 다른 컴포넌트 가져오기

```
public class PlayerController : MonoBehaviour
{
    Rigidbody2D rb;           // cached reference
    SpriteRenderer sr;

    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        sr = GetComponent<SpriteRenderer>();
    }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            rb.velocity = new Vector2(0, 5);
            sr.color = Color.red;
        }
    }
}
```

Update에서 GetComponent를 호출하지 마라

```
// BAD: searches every frame (60+ times per second)  
void Update() {  
    GetComponent<Rigidbody2D>().velocity = ...;  
}  
  
// GOOD: search once, reuse the reference  
Rigidbody2D rb;  
void Awake() { rb = GetComponent<Rigidbody2D>(); }  
void Update() { rb.velocity = ...; }
```

“찾는 것은 한 번, 사용하는 것은 매번”

→ Unity 스크립트의 가장 기본적인 성능 규칙

Vector3 — 3D 좌표/방향

```
Vector3 p = new Vector3(2f, 5f, 0f);

// Vector arithmetic
Vector3 c = a + b;           // (1, 1, 0)
Vector3 d = a * 5f;          // (5, 0, 0)

// Useful constants
Vector3.zero;                // (0, 0, 0)
Vector3.up;                   // (0, 1, 0)
Vector3.right;                // (1, 0, 0)
```

- x, y, z 세 float 값을 묶은 구조체
- transform.position = new Vector3(...)로 위치 통째로 설정
- Lecture 7의 PointF와 같은 발상
- 2D는 Vector2(x, y)

transform — 모든 GameObject의 필수 컴포넌트

```
public class Mover : MonoBehaviour
{
    public float speed = 3f;

    void Update()
    {
        // 3 units per second to the right
        transform.position += new Vector3(
            speed * Time.deltaTime, 0, 0);

        // 90 degrees per second around Z
        transform.Rotate(0, 0, 90f * Time.deltaTime);
    }
}
```

transform, gameObject는 MonoBehaviour가 이미 가지고 있는 속성.

Inspector와 직렬화

Inspector에 노출되는 필드

```
public class Mover : MonoBehaviour
{
    public float speed = 3f;           // shown
    public Color color = Color.red;    // shown
    public GameObject target;          // shown (drag-drop)

    private int hiddenValue = 10;      // NOT shown
}
```

- 디자이너가 코드 수정 없이 값을 조정 가능
- public 필드 → 자동 노출
- private 필드 → 노출 안 됨
- 프로퍼티(public int X { get; set; }) → 노출 안 됨

권장 패턴: [SerializeField] private

```
public class Player : MonoBehaviour
{
    // Editable in Inspector, but NOT accessible
    // from other scripts
    [SerializeField] private float speed = 3f;
    [SerializeField] private int maxHp = 100;

    // Read-only access from outside
    public int CurrentHp { get; private set; }
}
```

한 줄로 두 가지를 동시에 만족:

- Inspector에서 조정 가능 (디자이너 편의)
- 외부에서 접근 불가 (캡슐화)

주요 고려 사항

(1) MonoBehaviour는 new로 만들지 마라

```
// WRONG: compiles, but Unity ignores it
PlayerController p = new PlayerController();

// CORRECT: attach to a GameObject
GameObject obj = new GameObject("Player");
PlayerController p = obj.AddComponent<PlayerController>();

// ALSO CORRECT: instantiate from a Prefab
GameObject p2 = Instantiate(playerPrefab);
```

new로 만들면:

- Unity가 존재를 모름 → Update 안 호출
- transform이 null

AddComponent vs Instantiate vs Prefab

`AddComponent<T>()`

이미 존재하는 `GameObject`에 컴포넌트 하나 추가

`Instantiate(prefab)`

Prefab을 복제하여 Scene에 새 `GameObject` 생성

Prefab(프리팹)

재사용 가능한 `GameObject` 템플릿 (.prefab 파일)

복잡한 적 캐릭터를 한 번 만들어 두고 `Instantiate`로 무한 복제

- 단순 일회성 → `new GameObject + AddComponent`
- 복잡/반복 생성 → `Prefab + Instantiate`

(2) Update에서 무거운 작업을 하지 마라

매 프레임 호출되므로 다음은 피해야 함:

- `GameObject.Find(...)` — Scene 전체 검색
- `FindObjectOfType<T>()` — 모든 오브젝트 순회
- `GetComponent<T>()` — 컴포넌트 목록 검색
- `new SomeClass()` 반복 — GC 부담
- `Debug.Log(...)` 남발 — 의외로 비싸다

원칙: 찾는 것은 한 번, 사용하는 것은 매번

가비지 컬렉션(GC)이 게임에서 왜 문제인가?

문제: 프레임 끊김

GC는 메모리 청소를 위해 **프로그램을 잠시 멈춘다**.

60fps → 한 프레임 16ms뿐인데 GC가 5ms만 발생해도 끊김 발생.

원인: 반복적인 객체 생성

`new List<>()`, `new string()`을 매 프레임 만드는 것

해결

- 필드로 미리 만들어 두고 재사용
- `string` + 대신 `StringBuilder`
- 자주 생성/파괴되는 것 → **Object Pooling**

(3) Unity의 null은 일반 C#의 null과 다르다

```
GameObject obj = ...;
Destroy(obj); // marked for destruction

// One frame later:
if (obj == null) // TRUE (Unity reports null)
    Debug.Log("destroyed");

// But the C# reference is technically not null!
object asObj = obj;
if (asObj == null) // FALSE
    ...
```

실용적 규칙: 항상 `== null`로 비교하라.

?., ??는 “가짜 null”을 인식하지 못함.

(4) Coroutine — 시간이 흐르는 작업

```
using System.Collections;
using UnityEngine;

public class Bomb : MonoBehaviour
{
    void Start() { StartCoroutine(ExplodeAfterDelay()); }

    IEnumerator ExplodeAfterDelay()
    {
        Debug.Log("Bomb planted");
        yield return new WaitForSeconds(2f);
        Debug.Log("BOOM!");
        Destroy(gameObject);
    }
}
```

- yield return null — 다음 프레임까지
- yield return new WaitForSeconds(2f) — 2초 대기
- yield return new WaitUntil(...) — 조건까지

(5) 직접 그리지 않는다

Lecture 7: OnPaint에서 `g.FillRectangle`로 매 프레임 직접

Unity: 전용 컴포넌트가 자동으로 그린다

- 이미지 → `SpriteRenderer`
- 3D 모델 → `MeshRenderer`
- UI 텍스트 → `TextMeshProUGUI`
- 선/궤적 → `LineRenderer`
- 파티클 → `ParticleSystem`

스크립트는 “어디에/어떻게” 그릴지만 결정
(→ `transform.position`, `sr.color` 등을 변경)

Lecture 7과의 비교

Windows Forms 게임 vs Unity

	Lecture 7 (WinForms)	Unity
진입점	Main / Application.Run	없음 (엔진이 자동)
게임 루프	Timer.Tick 직접 작성	Update 자동
프레임 보정	Timer.Interval = 16	Time.deltaTime
화면 갱신	Invalidate + OnPaint	자동 렌더링
그리기	Graphics.FillRectangle	SpriteRenderer
객체 모델	클래스 직접 작성	GameObject + Component
입력 처리	OnKeyDown 오버라이드	Input.GetKey 폴링
충돌 판정	Math.Abs(...) < N	Collider + 콜백
값 조정	코드 수정 후 재컴파일	Inspector에서 즉시

Input.GetKey 세 가지 변형

메서드	언제 true
Input.GetKey(K)	키 눌려 있는 동안 매 프레임 (이동)
Input.GetKeyDown(K)	키 누른 그 프레임만 (점프, 발사)
Input.GetKeyUp(K)	키 뗀 그 프레임만 (차징 해제)

```
void Update()
{
    if (Input.GetKey(KeyCode.LeftArrow))
        transform.Translate(-5 * Time.deltaTime, 0, 0);

    if (Input.GetKeyDown(KeyCode.Space))
        Shoot();
}
```

흔한 실수: 점프에 GetKey 쓰면 매 프레임 점프 → 미친 듯이 튼

Collider와 충돌 콜백

Collider를 붙이면 충돌 검사를 **엔진이 자동으로** 수행.

```
void OnCollisionEnter2D(Collision2D c) {  
    // physical collision (bounces)  
    Debug.Log("Hit: " + c.gameObject.name);  
}  
  
void OnTriggerEnter2D(Collider2D c) {  
    // pass-through zone (no bounce)  
    Debug.Log("Entered: " + c.gameObject.name);  
}
```

- **Collision** — 실제로 부딪히고 튕김 (벽, 바닥)
- **Trigger** — 통과는 되지만 알림 받음 (아이템 영역)
- Is Trigger 체크박스로 모드 전환

“코드가 짧다”고 느끼는 이유:

- 진입점, 메시지 루프, 창 생성
- 프레임 타이머, 더블 버퍼링, 화면 갱신
- 키보드/마우스/터치 입력 저수준 처리
- 물리 시뮬레이션 (중력, 충돌, 마찰)
- 텍스처/메시 로딩, 셰이더, 렌더링 파이프라인
- 오디오, 네트워크, 파일 시스템 추상화

우리는 “내 게임의 규칙”만 쓰면 된다.

실전 예제: 충돌하면 합쳐지는 공

화면에 여러 공이 떠다니다가:

- 벽에 부딪히면 **튕김** (자동)
- 공끼리 부딪히면 **하나로 합쳐짐** (우리 코드)

물리 규칙

- 면적 보존: $r_{new} = \sqrt{r_1^2 + r_2^2}$
- 질량 보존: $m_{new} = m_1 + m_2$
- 운동량 보존: $\vec{v}_{new} = \frac{m_1\vec{v}_1 + m_2\vec{v}_2}{m_1 + m_2}$
- 위치는 **질량 중심**

Hierarchy 설계

```
Main Camera          // Unity default
GameManager          // empty GameObject
    + BallSpawner.cs  // our script
Walls                 // empty group
    + WallTop          // BoxCollider2D
    + WallBottom
    + WallLeft
    + WallRight
```

- 공(Ball)은 Hierarchy에 두지 않음
- Prefab으로 만들고 BallSpawner가 Instantiate
- 빈 GameObject Walls는 4개 벽을 묶어 정리

컴포넌트	역할
Transform	위치/크기 (자동)
SpriteRenderer	원 모양 표시
CircleCollider2D	원형 충돌 영역
Rigidbody2D	물리 (관성, 충돌 응답)
Ball.cs	우리 스크립트

Rigidbody2D 설정

- Gravity Scale = 0 (떨어지지 않게)
- Linear Drag = 0 (계속 움직이게)

Physics Material 2D Bouncy: Friction = 0, Bounciness = 1

스크립트 설계 — 역할 분리

Ball.cs — 개별 공의 동작

- 시작 시 무작위 방향 발사
- 다른 공과 충돌하면 합치기

BallSpawner.cs — 전체 게임 관리

- 시작 시 N개의 공 생성

충돌 시 “누가 합치는가”?

A↔B 충돌 시 OnCollisionEnter2D가 양쪽에서 호출됨.

→ GetInstanceID()가 더 작은 쪽만 처리.

Ball.cs — 초기화와 충돌 감지

```
public class Ball : MonoBehaviour
{
    [SerializeField] private float initialSpeed = 3f;
    private Rigidbody2D rb;

    void Awake() { rb = GetComponent<Rigidbody2D>(); }

    void Start()
    {
        float angle = Random.Range(0f, 360f) * Mathf.Deg2Rad;
        Vector2 dir = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle));
        rb.velocity = dir * initialSpeed;
    }

    void OnCollisionEnter2D(Collision2D c)
    {
        Ball other = c.gameObject.GetComponent<Ball>();
        if (other == null) return;           // hit a wall
        if (GetInstanceID() > other.GetInstanceID()) return;
        Merge(other);
    }
    // ... Merge() on next slide
}
```

```
void Merge(Ball other)
{
    float r1 = transform.localScale.x * 0.5f;
    float r2 = other.transform.localScale.x * 0.5f;
    float rNew = Mathf.Sqrt(r1*r1 + r2*r2);    // area conserved

    float m1 = rb.mass, m2 = other.rb.mass;
    float mNew = m1 + m2;                    // mass conserved

    Vector2 vNew = (m1*rb.velocity + m2*other.rb.velocity) / mNew;
    Vector2 pNew = (m1*(Vector2)transform.position
        + m2*(Vector2)other.transform.position) / mNew;

    transform.position = pNew;
    transform.localScale = Vector3.one * (rNew * 2f);
    rb.mass = mNew;
    rb.velocity = vNew;

    Destroy(other.gameObject);                // remove the other
}
```

```
public class BallSpawner : MonoBehaviour
{
    [SerializeField] private GameObject ballPrefab;
    [SerializeField] private int count = 10;
    [SerializeField] private Vector2 areaMin = new Vector2(-7,-4);
    [SerializeField] private Vector2 areaMax = new Vector2( 7, 4);

    void Start()
    {
        for (int i = 0; i < count; i++)
        {
            float x = Random.Range(areaMin.x, areaMax.x);
            float y = Random.Range(areaMin.y, areaMax.y);
            Instantiate(ballPrefab,
                        new Vector3(x, y, 0),
                        Quaternion.identity);
        }
    }
}
```

Editor 셋업: Step-by-Step (1/2)

Step 1: 새 2D 프로젝트

- Unity Hub → New Project → “2D Core”

Step 2: 벽 만들기

- Walls 빈 GameObject 생성
- 그 아래 2D Object > Sprites > Square 4개 (Top/Bottom/Left/Right)
- Scale/Position 조정 + BoxCollider2D 추가

Step 3: BallPrefab

- 2D Object > Sprites > Circle 생성 → Ball
- Rigidbody2D (Gravity 0) + CircleCollider2D 추가
- Bouncy Physics Material 2D 만들어 Collider에 할당
- Ball.cs 부착 → Project로 드래그하여 Prefab화

Editor 셋업: Step-by-Step (2/2)

Step 4: GameManager + Spawner

- 빈 GameObject GameManager 생성
- BallSpawner.cs 부착
- Inspector의 Ball Prefab 슬롯에 BallPrefab 드래그

Step 5: Play

- 상단 ► Play 버튼 클릭
- 10개의 공이 튕기다가 부딪히면 큰 공 하나로 합쳐짐

실제 작성한 코드는 약 60줄. Lecture 7 방식이라면 수백 줄 필요.

무엇이 어디서 일어나는가

이벤트	누가 처리하나
공 생성	<code>BallSpawner.Start</code>
초기 속도 설정	<code>Ball.Start</code>
매 프레임 위치 갱신	<code>Rigidbody2D</code> (자동)
벽 충돌 반사	<code>Collider2D + Material</code> (자동)
공-공 충돌 감지	<code>OnCollisionEnter2D</code> 콜백
합치기 계산	<code>Ball.Merge</code>
사라진 공 제거	<code>Destroy(...)</code>

우리가 작성한 부분 = 무작위 초기 속도 + 합치기 로직.
나머지는 모두 Unity 컴포넌트가 처리.

단계별 도전 과제

앞 예제를 발판으로 **직접 만들어 볼** 과제를 단계별로 제시.

Phase	내용
1. 예제 변형	색/사운드 등 시각·청각 피드백 추가
2. 새 메커니즘	코루틴, UI, 입력으로 규칙 변경
3. 새 게임	Rigidbody/Collider/Prefab 종합 활용
4. 자유 창작	5개 이상 개념을 활용해 완성

제출: 프로젝트 압축(Library/, Temp/ 제외) + README.md + 화면 캡처

Phase 1 — 예제 변형 (기초)

코드를 **조금만** 고쳐 결과가 크게 달라지는 단계.

1. 색상 평균 합치기

```
sr.color = (sr.color + other.sr.color) * 0.5f;
```

2. 속도에 따른 색 변화

```
Color.Lerp(Color.blue, Color.red, |v|/maxSpeed)
```

3. 충돌 사운드

```
AudioSource.PlayOneShot(wallHit / mergeHit)
```

벽인지 공인지 GetComponent<Ball>() == null로 분기

Inspector에서 즉시 반영되는 감각을 익히는 데 집중.

Phase 2 — 새 메커니즘 추가 (응용)

1. 마우스 클릭으로 공 추가

```
if (Input.GetMouseButtonDown(0)) {  
    Vector3 mp = Input.mousePosition;  
    mp.z = -Camera.main.transform.position.z;  
    Vector3 w = Camera.main.ScreenToWorldPoint(mp);  
    w.z = 0;  
    Instantiate(ballPrefab, w, Quaternion.identity);  
}
```

2. 최대 크기 분열

반지름 r $\geq$ maxRadius이면 둘로 쪼개기 (면적/운동량 보존)

3. 코루틴 자동 스폰

IEnumerator + yield return new WaitForSeconds(2f)

4. HUD 점수 (싱글톤 + TextMeshProUGUI)

GameManager.Instance.AddScore(1)

Phase 3 — 새 게임 (종합 응용)

과제 3.1 — 벽돌 깨기 (Block Breaker) “공 합치기” 자산(BallPrefab, Bouncy) 재사용.

- Paddle.cs — 좌우 키 이동
- Brick.cs — 충돌 시 Destroy(gameObject), 점수 +10
- Ball.cs — 화면 아래로 떨어지면 Game Over
- 모든 벽돌 사라지면 Stage Clear

과제 3.2 — 중력 토글 시뮬레이터

SPACE 키로 `Physics2D.gravity = -Physics2D.gravity;`

필수: README에 비교 글 작성

Lecture 7 “벽 반사” / “벽돌 충돌” 코드 줄 수 vs. Unity에서 우리가 작성한 줄 수. 사라진 코드가 어떤 컴포넌트로 옮겨졌는지 짚지어 설명.

Phase 4 — 자유 창작

배운 개념 **5개 이상**을 사용해 미니 게임을 자유 주제로 완성.

필수 개념 (5개 이상)

`[SerializeField]` / `GetComponent` 캐싱 / `Rigidbody2D` / `Prefab + Instantiate` /
`OnCollisionEnter2D` / `Coroutine` / `Time.deltaTime` / `TextMeshProUGUI`

주제 예시

- Catch the Star — 떨어지는 별 받기
- Mini Shooter — 적이 내려오고 플레이어가 발사
- Dodge — 떨어지는 공 피하기
- Top-Down Maze — 미로 탈출
- Flappy 스타일 — 클릭으로 점프, 장애물 회피

팁: 한 가지 메커니즘만 **확실히** 동작시킨 뒤 확장.

정리

이번 강의에서 배운 핵심 개념

개념	핵심 내용
MonoBehaviour	Unity가 인식하는 스크립트가 되기 위한 필수 상속
생명 주기	Awake/Start/Update/FixedUpdate/LateUpdate 호출 시점
컴포넌트 모델	상속이 아닌 조합 으로 GameObject 구성
GetComponent 캐싱	Update에서 매번 호출하지 말 것
Time.deltaTime	프레임 속도와 무관한 일정 속도
Inspector 직렬화	public / [SerializeField] 규칙
new 금지	AddComponent 또는 Instantiate로
가짜 null	항상 == null로 안전하게 검사
Coroutine	시간 흐름이 있는 작업 표현

1. Unity Editor에서 LifecycleDemo 스크립트를 붙이고 Console에서 Awake → Start 순서를 확인하라
2. Mover 스크립트를 작성하여 speed를 Inspector에서 조정해 보라
3. Lecture 7 슈팅 게임의 “플레이어 이동”만 Unity로 옮겨 보라
(Player.cs에서 키 입력 + transform.position 변경)
4. Bomb 코루틴을 변형하여 1초 간격으로 Debug.Log를 5번 출력 후 오브젝트가 사라지게 하라

질문?